Practical 5

Inheritance, Abstract Classes & Polymorphism

Part A — Practice Programs (Hour 1)

1) Shape Areas — abstract Shape and polymorphic loop

Code

```
public class Shapes {
    static abstract class Shape {
        abstract double area();
        public String toString() { return getClass().getSimpleName(); }
    }
    static class Circle extends Shape {
        double r;
        Circle(double r) { this.r = r; }
        double area() { return Math.PI * r * r; }
    }
    static class Rectangle extends Shape {
        double w, h;
        Rectangle(double w, double h) { this.w = w; this.h = h; }
        double area() { return w * h; }
    }
    static class Triangle extends Shape {
        double b, h;
        Triangle(double b, double h) { this.b = b; this.h = h; }
        double area() { return 0.5 * b * h; }
    }

    public static void main(String[] args) {
        Shape[] shapes = { new Circle(3), new Rectangle(4, 5), new Triangle(6, 7), new Circle(1.5),
            new Rectangle(2, 2) };
        double total = 0;
        Shape largest = shapes[0];
        for (Shape s : shapes) {
            double a = s.area();
            total += a;
            if (a > largest.area()) largest = s;
            System.out.printf("%-10s area = %7.2f   running total = %8.2f%n", s, a, total);
        }
        System.out.printf("Largest: %s (%.2f)%n", largest, largest.area());
    }
}
```

Output

```
Circle     area =   28.27   running total =    28.27
Rectangle  area =   20.00   running total =    48.27
Triangle   area =   21.00   running total =    69.27
Circle     area =    7.07   running total =    76.34
Rectangle  area =    4.00   running total =    80.34
Largest: Circle (28.27)
```

2) Payroll — abstract Employee, super and instanceof

Code

```
public class Payroll {
    static abstract class Employee {
        protected String name;
        protected int id;
        Employee(String name, int id) { this.name = name; this.id = id; }
        abstract double monthlySalary();
    }
    static class FullTime extends Employee {
        double fixed;
        FullTime(String name, int id, double fixed) { super(name, id); this.fixed = fixed; }
        double monthlySalary() { return fixed; }
    }
    static class PartTime extends Employee {
        int hours; double rate;
        PartTime(String name, int id, int hours, double rate) { super(name, id); this.hours =
            hours; this.rate = rate; }
        double monthlySalary() { return hours * rate; }
    }
    static class Intern extends Employee {
        double stipend;
        Intern(String name, int id, double stipend) { super(name, id); this.stipend = stipend; }
        double monthlySalary() { return stipend; }
    }

    public static void main(String[] args) {
        Employee[] staff = {
            new FullTime("Riya", 1, 60000),
            new PartTime("Aman", 2, 80, 250),
            new Intern("Neha", 3, 12000),
            new FullTime("Karan", 4, 45000)
        };
        double total = 0;
        for (Employee e : staff) {
            double pay = e.monthlySalary();
            total += pay;
            System.out.printf("%-6s (id %d) salary = %9.2f", e.name, e.id, pay);
            if (e instanceof Intern) System.out.print("   <- intern (stipend only, no benefits)");
            System.out.println();
        }
        System.out.printf("Total payroll: %.2f%n", total);
    }
}
```

Output

```
Riya   (id 1) salary =  60000.00
Aman   (id 2) salary =  20000.00
Neha   (id 3) salary =  12000.00   <- intern (stipend only, no benefits)
Karan  (id 4) salary =  45000.00
Total payroll: 137000.00
```

3) Library Late Fees (For Advanced Learners — home) — own hierarchy

Code

```
public class Library {
    static abstract class Media {
        protected String title;
        protected int daysLate;
        Media(String title, int daysLate) { this.title = title; this.daysLate = daysLate; }
        abstract double lateFee();
    }
    static class Book extends Media {
        Book(String t, int d) { super(t, d); }
        double lateFee() { return 2.0 * daysLate; }
    }
    static class Dvd extends Media {
        Dvd(String t, int d) { super(t, d); }
        double lateFee() { return Math.min(5.0 * daysLate, 50.0); }   // capped at 50
    }
    static class Magazine extends Media {
        Magazine(String t, int d) { super(t, d); }
        double lateFee() { return daysLate <= 3 ? 0 : 1.0 * (daysLate - 3); }  // 3 grace days
    }

    public static void main(String[] args) {
        Media[] batch = {
            new Book("Clean Code", 4),
            new Dvd("Inception", 12),
            new Magazine("Wired", 5),
            new Magazine("Time", 2),
            new Book("Effective Java", 0)
        };
        double total = 0;
        for (Media m : batch) {
            double fee = m.lateFee();
            total += fee;
            System.out.printf("%-8s %-15s late %2d days -> fee %6.2f%n",
                m.getClass().getSimpleName(), m.title, m.daysLate, fee);
        }
        System.out.printf("Total late fees: %.2f%n", total);
    }
}
```

Output

```
Book     Clean Code      late  4 days -> fee   8.00
Dvd      Inception       late 12 days -> fee  50.00
Magazine Wired           late  5 days -> fee   2.00
Magazine Time            late  2 days -> fee   0.00
Book     Effective Java  late  0 days -> fee   0.00
Total late fees: 60.00
```

Design chosen: Book pays 2.00 per day; Dvd pays 5.00 per day capped at 50; Magazine has 3 grace days then 1.00 per day.

Part B — Project Work (Hour 2): MiniBank

Account becomes abstract with interestRate() and canWithdraw(long). SavingsAccount, CurrentAccount and FixedDepositAccount each supply their own rules, and the rest of MiniBank handles all of them through the Account type.

1) Abstract Account with abstract interestRate() and canWithdraw(long)

Code

```
public abstract class Account {
    private final String accountNumber;
    private String ownerName;
    protected long balance;
    private boolean active = true;
    private static int counter = 0;

    protected Account(String ownerName, long openingBalance) {
        counter++;
        this.accountNumber = String.format("AC%04d", counter);
        this.ownerName = ownerName;
        this.balance = openingBalance;
    }

    public abstract double interestRate();
    public abstract boolean canWithdraw(long amount);

    public void deposit(long amount) {
        if (amount > 0) balance += amount;
    }

    public boolean withdraw(long amount) {
        if (amount > 0 && canWithdraw(amount)) {
            balance -= amount;
            return true;
        }
        return false;
    }

    // supplementary: monthly interest from interestRate() and balance
    public double monthlyInterest() {
        return balance <= 0 ? 0 : balance * interestRate() / 100 / 12;
    }

    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName() { return ownerName; }
    public long getBalance() { return balance; }

    public String toString() {
        return getClass().getSimpleName() + "[" + accountNumber + ", " + ownerName + ", balance=" +
            balance + "]";
    }
}
```

Supplementary problem covered: monthlyInterest() in Account uses interestRate() and balance.

2-5) SavingsAccount, CurrentAccount, FixedDepositAccount — each calls super(...)

Code

SavingsAccount.java

```
public class SavingsAccount extends Account {
    private long minBalance;

    public SavingsAccount(String ownerName, long openingBalance, long minBalance) {
        super(ownerName, openingBalance);
        this.minBalance = minBalance;
    }

    public long getMinBalance() { return minBalance; }

    public double interestRate() { return 4.0; }

    public boolean canWithdraw(long amount) {
        return balance - amount >= minBalance;
    }
}
```

CurrentAccount.java

```
public class CurrentAccount extends Account {
    private long overdraftLimit;

    public CurrentAccount(String ownerName, long openingBalance, long overdraftLimit) {
        super(ownerName, openingBalance);
        this.overdraftLimit = overdraftLimit;
    }

    public double interestRate() { return 0; }

    public boolean canWithdraw(long amount) {
        return balance - amount >= -overdraftLimit;
    }
}
```

FixedDepositAccount.java

```
public class FixedDepositAccount extends Account {
    public FixedDepositAccount(String ownerName, long openingBalance) {
        super(ownerName, openingBalance);
    }

    public double interestRate() { return 7.0; }

    public boolean canWithdraw(long amount) {
        return false;   // deposit is locked
    }
}
```

6) main — Account[] array, run-time polymorphism and pattern instanceof

Code

```
public class MiniBank {
    public static void main(String[] args) {
        Account[] accounts = {
            new SavingsAccount("Riya", 10000, 1000),
            new CurrentAccount("Aman", 5000, 2000),
            new FixedDepositAccount("Neha", 50000)
        };

        System.out.println("--- Run-time polymorphism: interestRate() ---");
        for (Account a : accounts) {
            System.out.println(a.getClass().getSimpleName() + " " + a.getAccountNumber() + " -> " +
                a.interestRate());
        }

        System.out.println("--- Withdrawal rules via canWithdraw() ---");
        System.out.println("Savings  withdraw 9500 -> " + accounts[0].withdraw(9500) + " (would
            break min balance)");
        System.out.println("Savings  withdraw 9000 -> " + accounts[0].withdraw(9000) + ", balance "
            + accounts[0].getBalance());
        System.out.println("Current  withdraw 6500 -> " + accounts[1].withdraw(6500) + ", balance "
            + accounts[1].getBalance() + " (overdraft)");
        System.out.println("Current  withdraw 1000 -> " + accounts[1].withdraw(1000) + " (beyond
            overdraft)");
        System.out.println("FD       withdraw 100  -> " + accounts[2].withdraw(100) + " (locked)");

        System.out.println("--- Pattern instanceof ---");
        for (Account a : accounts) {
            if (a instanceof SavingsAccount s) {
                System.out.println(s.getAccountNumber() + " is a SavingsAccount, minBalance = " +
                    s.getMinBalance());
            }
        }

        System.out.println("--- monthlyInterest() ---");
        for (Account a : accounts) {
            System.out.printf("%s %s: %.2f%n", a.getClass().getSimpleName(), a.getAccountNumber(),
                a.monthlyInterest());
        }
    }
}
```

Output

```
--- Run-time polymorphism: interestRate() ---
SavingsAccount AC0001 -> 4.0
CurrentAccount AC0002 -> 0.0
FixedDepositAccount AC0003 -> 7.0
--- Withdrawal rules via canWithdraw() ---
Savings  withdraw 9500 -> false (would break min balance)
Savings  withdraw 9000 -> true, balance 1000
Current  withdraw 6500 -> true, balance -1500 (overdraft)
Current  withdraw 1000 -> false (beyond overdraft)
FD       withdraw 100  -> false (locked)
--- Pattern instanceof ---
AC0001 is a SavingsAccount, minBalance = 1000
--- monthlyInterest() ---
SavingsAccount AC0001: 3.33
CurrentAccount AC0002: 0.00
FixedDepositAccount AC0003: 291.67
```

Package layout, build and tag

Code

```
minibank/
 |-- Account.java
 |-- CurrentAccount.java
 |-- FixedDepositAccount.java
 |-- MiniBank.java
 +-- SavingsAccount.java

javac *.java
java MiniBank
cd ..
git add .
git commit -m "Practical 5 - inheritance and polymorphism"
git tag lab-05
```

interestRate() prints 4.0 for the savings account, 0.0 for the current account and 7.0 for the fixed deposit: Java picks the subclass version at run time.

Key Questions / Analysis / Interpretation

1. What is the difference between an abstract class and a concrete class?

A concrete class is complete and can be instantiated with new. An abstract class may contain abstract methods (no body) and cannot be instantiated; it exists to be extended, and each concrete subclass must implement all the abstract methods. Account is abstract because a generic 'account' without a type has no interest or withdrawal rule.

2. What is dynamic method dispatch (run-time polymorphism)?

When a method is called through a superclass reference, Java decides at run time, from the actual object type, which overridden version to run. In the loop for (Account a : accounts) a.interestRate() runs the Savings, Current or FixedDeposit version depending on the object stored in the array, so the loop needs no if/else on the type.

3. What does the super keyword do in a subclass constructor?

super(...) calls the superclass constructor so the inherited fields (accountNumber, ownerName, balance) are initialised before the subclass adds its own. It must be the first statement of the constructor; if it is omitted, Java inserts a call to the no-argument super constructor, which fails to compile here because Account has none.